

EE180 Lab 3

Let's Build a Processor

Due: Wednesday, February 25th, 2026 (11:59 PM)

1. Introduction

In lab 2, you saw some of the benefits of writing efficient, optimized software that runs on a general-purpose processor. In this lab, you will venture further: You will build a working 5-stage pipelined MIPS processor that is capable of executing MIPS instructions. You will be using Verilog, a popular hardware description language (HDL) to design this processor. You will be programming the FPGA on the board with the bitstream generated from your Verilog. At the end of this lab, you will be able to compile the Sobel filter code into MIPS assembly and run it on your processor to do edge detection on a video input!

2. Requirements

We will provide you with a five-stage pipelined processor that already has the pipeline registers inserted. You will need to augment this processor to implement a subset of the MIPS instruction set, add hazard detection, provide forwarding, and stall when data cannot be forwarded.

We provide tools that enable you to write C code and MIPS assembly and then compile and run it on your design. Most of your development will be in a test-driven simulation environment. We provide several tests to get you started, but we will be grading your design with a much larger set of tests. When you are confident with your implementation, you will be able to synthesize and run the Sobel filter from lab 2 on your processor in the FPGA of your development board.

In this lab, you will be required to:

1. Implement the following MIPS instructions:
 - a. `add`, `addi`, `addiu`, `addu`, `and`, `andi`, `beq`, `bgez`, `bgtz`, `blez`, `bltz`, `bne`, `j`, `jal`, `jalr`, `jr`, `lb`, `lbu`, `lh`, `ll`, `lui`, `lw`, `movn`, `movz`, `mul`, `nor`, `or`, `ori`, `sb`, `sc`, `sh`, `sll`, `sllv`, `slt`, `slti`, `sltiu`, `sltu`, `sra`, `srav`, `srl`, `srlv`, `sub`, `subu`, `sw`, `xor`, `xori`
2. Some of these instructions are already fully implemented, some are partially implemented, and some are not yet implemented at all. Implementing an instruction means that it works correctly in the processor and that you have provided a test to demonstrate it.
3. Add correct hazard detection, forwarding, and stall control to the processor. Specifically, your processor should never use stale data in the presence of a RAW hazard, and it should never stall unless it has to in order to resolve a RAW hazard. You should create tests to demonstrate that this works.
4. Demonstrate your design by running a Sobel filter program using your MIPS processor to process a video.

3. Starter Code

Obtain the starter code from GitLab. See Section 5 for more details.

The relevant components of the starter code are organized as follows:

- **README:** The project report that you will write.
- **make_submission.sh:** A script that packages your code, tests, and report for submission.
- **hw/:** MIPS Verilog source code and Zedboard files.
 - **Makefile:** Makefile for building the Zedboard image
 - **hdl/verilog/mips/:** MIPS Verilog source code
- **sim/:** Simulation and test infrastructure
 - **Makefile:** Test driver
 - **tests/:** MIPS unit tests
 - **sobel_asm/:** Sobel source code and test (MIPS assembly)
 - **sobel_c/:** Sobel source code and test (C code)
- **tools/:** GCC cross-compiler and utilities (optional)

4. Processor Model

It is important that you study the processor model we have provided before you begin coding. This model is slightly different from the approach in the textbook. It is divided into five pipeline stages contained within the following modules:

- **Instruction Fetch:** `instruction_fetch.v`
- **Instruction Decode:** `decode.v` and `regfile.v`
- **Execute:** `alu.v`
- **Memory:** `mips_cpu.v`
- **Writeback:** `regfile.v`

To make the code more readable, we have used a naming convention, which we encourage you to follow. Appended to each signal name in `mips_cpu.v` is the name of the stage in which the signal is used (or generated, in cases where stages must communicate with each other). For example, the signal which determines whether the instruction currently in the EX stage will involve writing data to memory is called `mem_we_ex`.

There are three important changes to note. First, unlike in the textbook, the ALU operands are chosen in the ID stage, not in the EX stage. As a result, there is no `alu_src` signal, and all forwarding logic that you create will need to appear in `decode.v`.

The second change arises from the fact that jumps and branches need to be resolved in the ID stage in order to avoid multiple stalls in the pipeline. Instead of using the ALU to resolve branch conditions, the register comparisons occur in the ID stage.

The third change is that all memory reads are delayed by one cycle. This means that while instructions are requested in the IF stage (via the program counter), the instruction is not available until the ID stage on the next cycle and therefore does not need to be saved in the ID pipeline register. Similarly, data memory requests begin in the EX stage and are available one cycle later in the MEM stage.

4.1 Load Linked / Store Conditional Semantics

For the purposes of this lab, you can implement the following simplified behavior for the load linked (`ll`) and store conditional (`sc`) instructions:

- **sc:** Store a word to memory if and only if an `ll` instruction preceded it and no other stores (of any type) were executed in between the `ll` and `sc` instructions. If the store is successful, write a '1' to register `rt`.

Otherwise, if the store is not successful, write a '0' to register `rt`. The memory address is computed in the same way as the regular `sw` instruction.

- `ll`: Similar to load word (`lw`), but additionally initializes the state that allows the `sc` instruction to operate correctly.

4.2 Verilog Code

The model we have provided does not yet handle hazards. A good approach to doing this lab is to use test programs that have sufficient nops between dependent instructions and first test for the correct functionality of the program. As you add support for forwarding and stalling, remove the appropriate nops and check that your program still works. Several of the test programs in the starter code demonstrate how to do this.

We have not provided all of the signals that you will need for hazards and instructions. It is up to you to decide what signals are needed. Beware: this part can get more complex than necessary. Try to think of a way to detect forward and stall conditions with as little logic as possible. Remember that for forwarding and stalls you should only have to edit *decode.v* and the portion of *mips_cpu.v* where the decode module is instantiated, since all forwarding and stalling occurs from the ID stage.

For this lab assignment, **we will have one branch delay slot**. This means that the instruction following any branch or jump will be executed before the target of the branch or jump is issued.

Note that the pipeline registers between the IF and ID stages have their own special enable signal, `en_if`, and the registers between the ID and EX stages have their own special reset signal, `rst_id`. In order to allow for stalling, the starter code assigns these signals using the stall signal. Make sure you understand why these signals are set the way they are.

You will be making several additions in this lab, which require much thought. Before jumping in and making changes, make sure you know what you are doing. When you make changes, thoroughly test your implementation before proceeding with new changes. We strongly encourage you to create a test before implementing the corresponding change in the hardware.

4.3 Memory Layout

The processor is connected to a limited amount of memory that is divided into several special regions. The instruction memory is a 16-KiB region that begins at address 0x0. This memory is read-only and the processor will begin execution at address 0x0 after a reset.

The data memory port is divided into several regions described in Table 4.1.

The stack/heap memory is a general-purpose read/write region that you can use for storing variables or creating stack frames.

The input/output buffer region is used to interface your processor with the ARM cores on the Zedboard when you are running your Sobel code. A memory read request to this region is serviced by the input buffer, while a memory write to this region is sent to the output buffer (this means a load following a store to an address in this region will not return the stored value).

There are three memory-mapped test registers to aid with simulation and coordination with the Zedboard. The simulation section describes this in more detail, but generally you will not need to think about any of the memory sections except the stack/heap region.

Address Start	Address End	Size	Description
0x00000000	0x00003FFF	16 KiB	MIPS stack/heap
0x00004000	0x7FFFFFFF	—	Reserved
0x80000000	0x8001FFFF	128 KiB	Input/Output buffers
0x80020000	0x80020003	4 Bytes	Status register
0x80020004	0x80020007	4 Bytes	Test register
0x80020008	0x8002000C	4 Bytes	Command register
0x8002000C	0xFFFFFFFF	—	Reserved

Table 4.1: MIPS data memory map

5. Setup Instructions

It is **highly recommended** you work on the FarmShare machines (through ssh on terminal or OnDemand). All software needed for the simulation part of this assignment is installed on these machines. See Section 7 for information regarding bitstream generation. You can get the files from GitLab

To connect to the FarmShare machines, you should log in using this command:

```
ssh -Y <sunet_id>@login.farmshare.stanford.edu
```

Clone the assignment from GitLab into your directory:

```
git clone https://code.stanford.edu/ee180/lab3.git
```

Now the code is ready to run and no further setup is required. If you are synthesizing your design, there are more setup instructions (see Section 7).

5.1 Running from Home

If you are using FarmShare, you can skip this section.

Alternatively, you may be able to do your design testing and simulation on your own computer instead of using FarmShare. For your own machine you will need a unix-like environment, and you will first need to install iverilog, which simulates Verilog designs and gtkwave which is a waveform viewer (the FarmShare machines already have this software).

After installing iverilog and gtkwave, you will need a MIPS cross-compiler to compile C and MIPS assembly code into instructions that can run on your processor. Unpack the starter code to your computer and run the following commands from within the starter code directory:

```
cd tools/gcc-mips
make -j4
```

This will download and build a GCC compiler within that directory. Note this process also requires GNU `sed` and `gcc` to compile the software (Mac OSX tends to use FreeBSD `sed` and Clang). This will take a while, but it only needs to be done once. (Use the option `-jn` to speed up the build process by using `n` threads.) When this process completes, you will have a MIPS cross-compiler in

```
tools/gcc-mips/mips_tc
```

After building the compiler, make the utility program by running the following commands (again from the starter directory):

```
cd tools/util/make_hex
make
```

Finally, you will need to modify the simulation infrastructure to point to your local MIPS compiler. Open `sim/Makefile` from your starter code directory. In the “Basic Settings” section, set `TST_TOOLCHAIN` to `../tools/gcc-mips/mips_tc`. Also set `TST_UTIL` to `../tools/util/make_hex`.

6. Simulation Infrastructure

The simulation environment allows you to create C and MIPS assembly programs that can automatically be discovered, compiled, and run on your processor. The simulation tools require a MIPS compiler, iverilog, and gtkwave. If you are running on FarmShare then all of these packages are installed. If you are running on your own machine, you need to make sure these are available.

The starter code comes with over 30 tests. To run all of them, go into the `sim/` directory and type

```
make test
```

This command will search the `tests/` directory for any existing subdirectories. Each subdirectory is the name of a test and contains source code and simulation configuration files for that particular test. For each discovered test, another Makefile is called (located in `harness/Makefile_MIPS`) which discovers the source code files in the test directory and compiles them with the GCC MIPS compiler. The output of this process is a `build/` directory within each test, which contains the following files:

- **app.bin**: The compiled program in binary form.
- **app.hex**: The compiled program in ASCII hex format.
- **app.lst**: An assembly code listing of the compiled program.

The `app.hex` file is loaded as the instruction memory for a particular test. You will find `app.lst` to be especially useful while debugging because it shows the instructions and their locations in memory.

When `make test` finishes building each of the tests in the `tests/` directory, it will aggregate each test’s result and display a report which lists each test and displays whether the test passed or failed. When you change the hardware design (Verilog) files or the test source code, running `make test` again will automatically rebuild any outdated components and run those tests again.

To run a single test, prefix the test name with `test_`. For example, to test `xor` you would run

```
make test_xor
```

To debug a test with the waveform viewer, prefix the test name with `wave_`. For example, to view the waveform of the `bne` test, run

```
make wave_bne
```

Note that if you are working remotely over SSH, you will need to enable X forwarding (`ssh -Y`) to view the waveform.

To add a new test, create a folder in the `sim/tests/` directory with the name of the new test (it is probably easiest to copy an existing test and modify it for your needs). The test infrastructure will automatically include the new test the next time you run it.

Every test generates either a “pass” (0x1) or “fail” (0x0) value and stores it to a special memory location at address 0x80020004. This location is a memory-mapped test register that is used only for this purpose. After storing the test result, the test terminates by storing a “done” value (0x1) to another special memory location at address 0x80020000. This location is a status register that will terminate the simulation when the “done” bit is set. Any tests you create will need to follow this pattern. Refer to an existing test to see how it’s done in assembly.

It is important to note that the supplied tests are only a starting point for your design. They do not cover all of the functionality that you will need to implement and are not guaranteed to even work in all situations. It is up to you to modify and add as many new tests as are sufficient for meeting the requirements of this lab.

When you have finished the processor implementation and all tests pass, you are ready to run the Sobel test. This is an assembly program that was compiled from C code. It is located in `sim/sobel_asm/`. This test will execute a few million instructions and usually takes several minutes to complete. For this reason it is outside of the usual test directory. To run the Sobel test, move the `sobel_asm` directory into `sim/tests/` and run `make test`.

You can also find the C code version of the Sobel test in `sim/sobel_c/`. You are not required to run this test, but may find it useful to help see what is going on in the assembly version or if you decide to optimize it for extra credit. Because this version is written in C, there is no way to guarantee which instructions will be emitted by the compiler. After you have moved it to the test directory and built it (via `make test`), you can enter the `sobel_test_c` directory and run `inst_check.sh`. This is a handy script that scans the compiled binary to see if any instructions are being used that your processor does not support (edit the script to define valid instructions for your design). If this happens, you have three choices: Modify the C code, hand-tune the output executable, or implement the needed instructions in your processor. Please note again that the C-based Sobel test is **not required** for this lab and is only there to help you with the algorithm and extra credit.

When the Sobel assembly test passes in addition to all of your own tests, you are ready to synthesize your design and try it out with the `baxter.avi` video.

7. Running Your Design on the Board

In order to see your MIPS processor in action, you should generate a bitstream from your code and use the bitstream to program the FPGA. Generating a bitstream involves a compilation-like process called logic synthesis. The generated bitstream file is then copied over to the board, where it is used to program the FPGA. The following sections provide more information on how to do this.

NOTE: You should attempt to test your design on the board **only after all tests pass in simulation**. Logic synthesis is a time-consuming process. Also, you have almost zero visibility into the individual design signals when running on the board. Synthesizing your design without passing simulation tests is a bad idea, and you will waste a considerable amount of time on it unnecessarily.

7.1 Logic Synthesis

Logic synthesis is the process by which a hardware design description – like your MIPS processor – is turned into a design implementation in terms of logic gates. This process involves multiple synthesis tools that convert your design HDL into a binary file called bitstream. To the end user like you, the flow resembles a (fairly slow) compilation-like process. This process will take 30 minutes to 1 hour to complete.

You have been provided with a Makefile interface to generate the bitstream located in the `hw/` directory.

Please use **FarmShare** for this. Once you have gotten onto one of these machines:

- Set up the environment:
 - `/farmshare/home/classes/ee/admin/software/bin/rhel8.sh`
 - `module load base/1.0`
 - `module load vivado/2019.2`
- Copy your project (if it's not already there) into your home directory ~
- Navigate to the `hw/` directory containing the Makefile and type `make`. This process will take a very long time and will create `fpga.img`. You should check the console output for any inferred latches or timing violations.
- To clean up files from an old build, type `make clean`.

If you run into any errors in synthesis, search for any error keywords in your directories. There will likely be a log file that contains the error it ran into.

7.2 Programming the FPGA

Once the bitstream is generated, programming the FPGA is a simple two-step process:

1. Copy `fpga.img` to your Zedboard using `scp`.
For example: `scp fpga.img user@ee180-40z:<path_to_dest>`
2. From your Zedboard, run the `prog_fpga` command:
`prog_fpga fpga.img`

Programming the FPGA can take a few seconds to complete. During the programming process, you can observe that the blue LED on the Zedboard turns off (if you're physically present). Once the programming has completed successfully, the blue LED turns back on again.

7.3 Running Your Processor

We have installed a binary on your board named `lab3` which can be found in `/usr/bin/`. You should be able to run this binary from any folder.

The generated executable `lab3` accepts numerous options. We have highlighted a few use cases below:

1. Your MIPS program binary with a specific input buffer:
`lab3 -p your_prog.hex -i inbuf.hex`
2. Your MIPS program binary with a specific input buffer, dump output buffer:
`lab3 -p your_prog.hex -i inbuf.hex -o outbuf.hex`
3. Your MIPS program with input from a video file (designed for Sobel, runs your processor in a loop):
`lab3 -p app.hex -f <path_to_file>`

Make sure the 3rd command works with a video file (such as the provided `baxter.avi`), as that is what we will be using to evaluate your code.

To run the 3rd command, `scp` the `baxter.avi` and `app.hex` files over to your Zedboard.

```
scp baxter.avi sim/tests/sobel_asm/build/app.hex user@ee180-40z:<path_to_dest>
```

Note that `app.hex` here is the output from making the `sobel_asm` test. Please see note from Section 6 on how to generate `app.hex`:

When you have finished the processor implementation and all tests pass, you are ready to run the Sobel test. This is an assembly program that was compiled from C code. It is located in `sim/sobel_asm`. This test will execute a few million instructions and usually takes several minutes to complete. For this reason it is outside of the usual test directory. To run the Sobel test, move the `sobel_asm` directory into `sim/tests` and run `make test`.

8. Extra Credit: Feeling Adventurous?

In lab 2, you were presented with the task of optimizing a Sobel implementation in C running on a fixed dual-core ARM processor. The only thing you could change was software code. In this lab you can change the software (Sobel) at the C level or at the assembly level. You can also change the hardware (MIPS processor).

Once you have implemented the baseline solution, you can implement additional features that can speed up the frames per second (fps) for extra credit. See the grading section for more details regarding the grading rubric.

Here are some suggestions that may help. Feel free to think of your own too, or discuss these with the TAs.

1. Optimize Hardware

- Implement more helpful instructions – need not be MIPS
- Execute more than one instruction at a time - look into [superscalar processors](#)
- [Out of Order Execution](#) - some dependent instructions end up having to stall, so instead of doing nothing, allow other non-dependent instructions to move forward first
- <Your idea here>

2. Optimize Software

- Standard lab 2 tricks - note that some will not apply
- Better instruction scheduling if possible
- <Your idea here>

You need to show an appreciable speedup from your optimizations in order to receive extra credit. We would love to hear your ideas as well. So talk to the TAs about what you have in mind!

9. Suggested Workflow

This lab demands a significantly higher amount of work compared to your previous labs. We recommend that you establish your workflow based on the following guidelines:

- Version and backup your code regularly: Maintain a private Git repository on github or bitbucket. Also, do not rely on anything not on the AFS space to be persistent.
- Start early: It is all too easy to underestimate the amount of work.

- Get comfortable with waveforms: You will be staring at waveforms for quite a bit in this lab using GTKWave during debugging.
- Code – Test in Simulation – Code: While your final design must run on the board, simulation offers substantially faster turnaround times during coding or debugging. Logic synthesis is a slow process and each build takes minutes to run, compared to a few seconds for simulation. A wise idea is to try synthesis only if all tests are passing in simulation.
- Think of the block diagram: Before starting out to write Verilog, make sure you clearly understand what you are implementing. If you cannot implement something on a drawing board, chances are you cannot implement it in Verilog because you don't yet fully understand all of the parts.
- Write new test cases: Think of ways by which you can stress test your processor. Every good test written will save you time in the future.
- Test often: Implementing one piece of functionality could break another. Running your tests often can help catch such issues quickly
- Enumerate all the cases of stalling/forwarding that is possible within the processor and ensure that there are tests for them. Our provided tests do not cover all of these behaviors and it is up to you to write some tests to exercise this behavior.
- Do not wait until the last day to run logic synthesis. You have to make sure that your design meets all the timing constraints. Waiting until the last day means that you have less time to debug any last-minute timing and/or board-related issues.

10. Resources

- Verilog reference guides and tutorials:
 - Stuart Sutherland's Verilog 2001 reference guide
http://www.sutherland-hdl.com/online_verilog_ref_guide/verilog_2001_ref_guide.pdf
 - ASIC World Verilog Tutorial
<http://asic-world.com/verilog/index.html>
 - HDL Bits Verilog Tutorials
https://hdlbits.01xz.net/wiki/Problem_sets#Verilog_Language
- Reference material on iverilog:
 - http://iverilog.wikia.com/wiki/User_Guide
- Reference material on GTKWave:
 - <http://gtkwave.sourceforge.net/gtkwave.pdf>

11. Grading

1. Base requirements (100%)
 - a. Autograder Tests (90%)
 - i. Your additional custom tests passing (10%)
 - ii. Provided tests passing (40%)
 - iii. Our custom tests passing (40%)
 - b. README (10%) containing the following:
 - i. Partner and group names
 - ii. List of instructions implemented
 - iii. Description of changes made to the control and datapath to handle hazards and forwarding
 - iv. If you used AI tools, explain how you use it to help with the assignment
 - v. (Extra credit only) Description of optimizations made to both software and hardware
2. Extra credit (20%)
 - a. Optimize Sobel! (20%)

Note: Extra credit will be awarded only if the base requirements are also met. Work for extra credit will NOT be regarded if your design fails to meet all base requirements listed above.

12. Submission

In order to create a submission tarball, run `source make_submission.sh` in your top-level directory. The submission zip file will be named `lab3_submission.zip`. This tarball contains all your MIPS Verilog source files, all test files and your README. Please submit your zip on GradeScope by 11:59 PM under the Lab 3 assignment. Be sure to submit as a group (you can add two people to one submission on GradeScope). There will be an autograder on GradeScope. This autograder may take a bit of time.